

class06

Alishba

```
add_2 <- function(x, y) {  
  # Adding variables x and y  
  x + y  
}
```

Testing my function:

```
add_2(5, 7)
```

```
[1] 12
```

Q1. Write a function `add()` to add some numbers together. Make sure to add comment(s) to the function explaining why you do things.

a. Your first version of the function should add two input numbers together; e.g. `add(4, 7)` should return 11.

```
4 + 2
```

```
[1] 6
```

```
x <- 4  
y <- 2  
x + y
```

```
[1] 6
```

```
add_2 <- function(x, y) {  
  #adding variables x and y  
  x + y  
}
```

b. Your second version of the function should add any numbers together that the user inputs; e.g. `add(1, 2, 3, -4)` should return 2.

```
add_multi <- function(x, ...) {  
  # Using sum because adding multiple numbers  
  sum(x, ...)  
}  
  
add_multi(1,2,3,-4)
```

```
[1] 2
```

Q2. Write a function `generate_dna()` that generates a random DNA sequence of a length input by the user; e.g. `generate_dna(11)` may return “ATTTAAGGCTG”. Make sure to add comments to the function. Play with functions `sample()` and `paste()` in your R-Studio Console to see how they can help here.

```
generate_DNA <- function(x) {  
  # generate a nucleotide vector  
  nucs <- c("A", "G", "T", "C")  
  # using sample to pull out random nucleotides  
  nucs_vector <- sample(nucs, x, replace=TRUE)  
  # Generating a DNA string from nucs_vector  
  paste(nucs_vector, collapse="")  
}  
  
# testing my generate_DNA() function  
generate_DNA(34)
```

```
[1] "CGCCGTTTTTTACCCGACATGAGTCGGTGCACTG"
```

Q3. Write a function `generate_protein()` that generates a random peptide/protein sequence of a length input by the user; e.g. `generate_protein(6)` may return “WQRTAG”. Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids can appear more than once. Make sure to add comments to the function.

```

generate_protein <- function(x) {
  # generate a protein vector
  protein <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S",
"V")
  # using sample to pull out random amino acids
  protein_vector <- sample(protein, x, replace=TRUE)
  # generating an amino acid string from protein_vector
  paste(protein_vector, collapse="")
}

# testing my generate_protein() function
generate_protein(6)

```

```
[1] "SAVAFL"
```

Q4. Use your `generate_protein()` function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function `sapply()` for this.

```

# I defined the four different lengths to generate
peptide_lengths <- c(5,7, 9, 11)
# I generated strings of 5, 7, 9, and 11 amino acids using sapply()
sapply(peptide_lengths, generate_protein)

```

```
[1] "VTCGY"      "SRTWGSA"    "QPMSTLTRE"  "YAITGAMCWYE"
```

Q5. Take each of your peptides from Q4 and run a BLASTP search (<https://blast.ncbi.nlm.nih.gov/Blast.cgi>) selecting the Non-redundant protein sequences (nr) database and homo sapiens (taxid: 9606) organism. For each search, before clicking BLAST check the Show results in a new window tab next to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. Once each search is done, for the top aligned protein for each peptide, list the percent identity (Per. Ident) times coverage (Query Cover) (for example, in the example below, 100% Query Cover times 85.71% Per. Ident = 100% x 85.71% = 1.00 x 0.8571 x 100% = 85.71%); make sure to type out your calculations in your Quarto document [note: if any of your values are below 30% you are probably miscalculating something]. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see Data Sheet link in the Canvas assignment). Also, for each datapoint, add your initials

in the “Your_Initials” column of the data sheet. Before adding your initials, scan through other used initials; if anyone has the same initials as you, modify your initials to make them unique (your initials will be used below to identify your specific data points in a ggplot graph).

5 peptide chain (LYPPF). (Per Ident) x (Query Cover) = 1.00 x 1.00 x 100% = 100%

7 peptide chain(VPTICKS). (Per Ident) x (Query Cover) = 1.00 x 0.71 x 100% = 71%

9 peptide chain (KMKPGPAKA). (Per Ident) x (Query Cover) = 0.8571 x 0.78 x 100% = 66.85%

11 peptide chain (LTYPKDWELFS). (Per Ident) x (Query Cover) = 0.8571 x 0.64 x 100% = 54.85%

Q6. Once there is a good amount of data in the shared excel sheet, export the sheet as a CSV UTF-8 file (Export option under the File tab) and save the file to your class working folder (i.e. the folder in which you are currently generating a Quarto document). In R-Studio, upload the csv file into a data frame using the command `peptide_df <- read.csv(“your_filename”).` Use the `head()` function to list the first 10 rows (note that the default for `head()` is 6) of the `peptide_df` dataframe as a test that everything looks good.

```
# exported csv file into a data frame
peptide_df <- read.csv("BIMM143_peptide(Sheet1).csv")

# listed the first 10 rows of my df
head(peptide_df, n=10)
```

	Length_aa	Max_Identity_perc	Your_Initials	X	X.1	X.2	X.3	X.4	X.5	X.6	X.7	X.8
1	5	100	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
2	7	85.71	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
3	9	66.85	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
4	11	55	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
5	5	80	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
6	7	85.71	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
7	9	66.85	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
8	11	64	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
9	5	100	KET	NA	NA	NA	NA	NA	NA	NA	NA	NA
10	7	85.71	KET	NA	NA	NA	NA	NA	NA	NA	NA	NA
	X.9	X.10	X.11	X.12	X.13	X.14	X.15	X.16	X.17	X.18		
1	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
2	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		

3	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
4	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
5	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
6	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
7	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
8	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
9	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
10	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

Q7: Use `ggplot2` to generate a box plot with `Length_aa` on the x-axis and `Max_Identity_perc` on the y-axis. Since the box plot will require categorical values for `Length_aa`, you may have to specify the x-axis as `as.factor(Length_aa)`. Feel free to pretty up the plot by adding your own x/y-axis labels, graph title, etc using the `labs()` function and/or using your favorite theme using a `theme()` function (see the [data-visualization_ggplot.pdf](#) cheat sheet in Canvas for options). Optional: you will reuse this plot a few times below, so your subsequent coding will be simpler if you assign your box plot to an object such as 'p' (`p <- ggplot(peptide_df) +....etc`).

```
# loaded ggplot2
library(ggplot2)

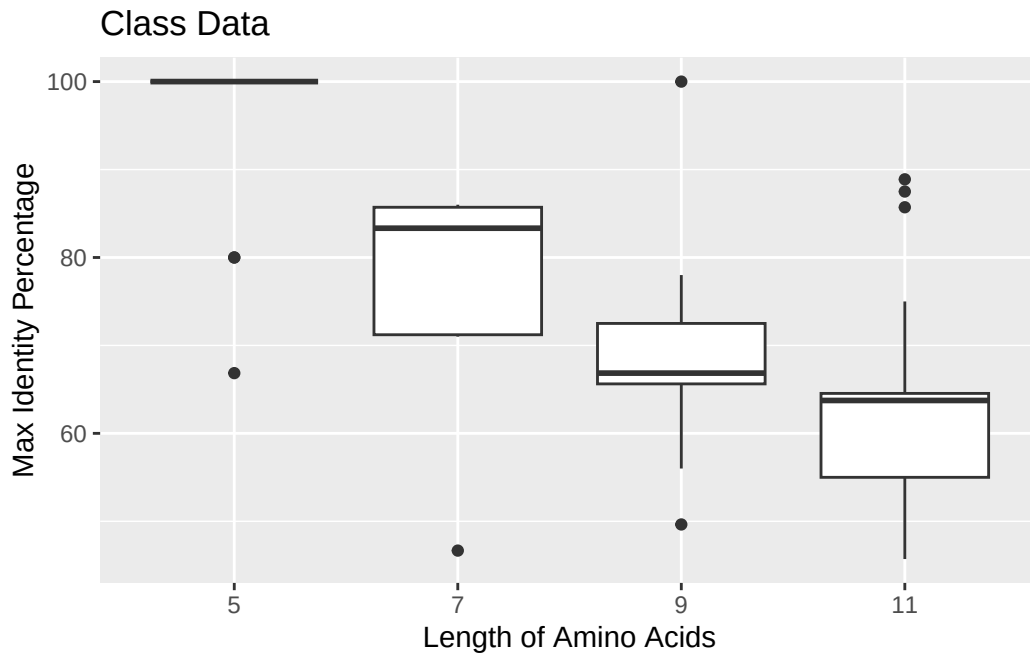
# changed y-axis to numerics
peptide_df$Max_Identity_perc <- as.numeric(peptide_df$Max_Identity_perc)
```

Warning: NAs introduced by coercion

```
# created boxplots of each amino acid length
p <- ggplot(peptide_df, aes(x = as.factor(Length_aa), y = Max_Identity_perc)) +
  geom_boxplot() +
  labs(
    title = "Class Data",
    x = "Length of Amino Acids",
    y = "Max Identity Percentage")

p
```

Warning: Removed 31 rows containing non-finite outside the scale range (``stat_boxplot()``).



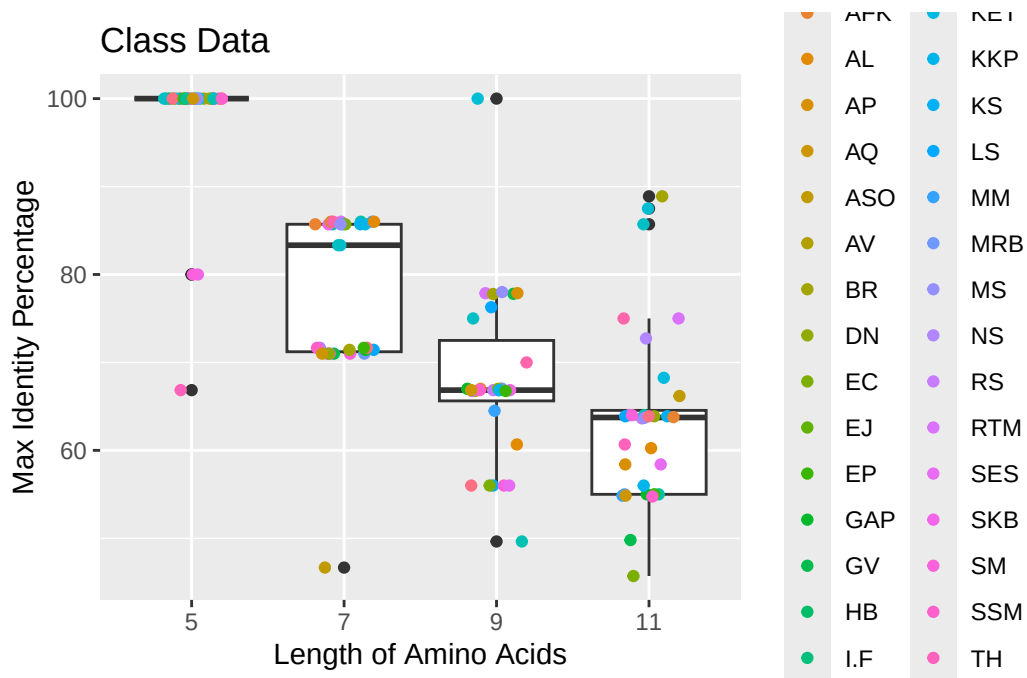
Q8. Let's add a `geom_jitter()` layer to your box plot to visualize the individual data points on top of the box plot and highlight your own data points.

a. First, add a `geom_jitter()` layer to your box plot with each data point colored according to students' initials by adding the following additional line to your plot: `+ geom_jitter(aes(col=Your_Initials))`. Feel free to narrow the spread of the jitter points using the `width` argument (e.g. `width = 0.2`). You can find your own datapoints this way via the color legend, but it looks rather messy.

```
# color coded each point
p + geom_jitter(aes(col=Your_Initials), width=0.2)
```

Warning: Removed 31 rows containing non-finite outside the scale range (``stat_boxplot()``).

Warning: Removed 31 rows containing missing values or values outside the scale range (``geom_point()``).



b. Now let's control the coloring ourselves to better highlight your own data points. We can do this by generating a vector with a color for each datapoint (we will use your favorite color for your datapoints and "GREY" for all other datapoints). You can then use this vector to assign colors with the `geom_jitter()` call.

i. First, use the `rep()` function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the `peptide_df` dataframe. Assign this vector to an object you name `color`: i.e. `color <- c(rep(,))` (fill in the blanks). Once you have generated the `color` object, use the `table()` function to ensure that you have the correct number of instances of "GREY" in `color`.

```
# created vector for mention of "GREY" for each row in dataframe
color <- rep("GREY", dim(peptide_df)[1])

table(color)
```

```
color
GREY
160
```

ii. Next, use the `Your_Initials` column in the `peptide_df` dataframe (which can be called as `peptide_df$Your_Initials`) to generate a `TRUE/FALSE` vector indicating which datapoints are yours (`TRUE`) and which are not (`FALSE`). Assign this vector to an object you name `mydata`: i.e. `mydata <- _____` (fill in the blank). Again, check that you have the correct number of `TRUE` and `FALSE` in `mydata` using the `table()` function.

```
# created vector to assign only my initials to TRUE
mydata <- c(peptide_df$Your_Initials == "AQ")

table(mydata)
```

```
mydata
FALSE  TRUE
  156     4
```

iii. Now, use the `mydata` vector to replace the “GREY” values in the `color` vector that correspond to your data points with your favorite color, like this: `color[mydata] <- “COLOR”` (replace “COLOR” with the name of your favorite color; see link at the bottom of the page to a webpage with 657 R color names to select a cool color). Note, this will replace “GREY” with your favorite color for all the positions that are `TRUE` in `mydata`, i.e. the positions of your data points. Again, check the vector with `table()`.

```
# changed color of all my data point
color[mydata] <- "darkred"

table(color)
```

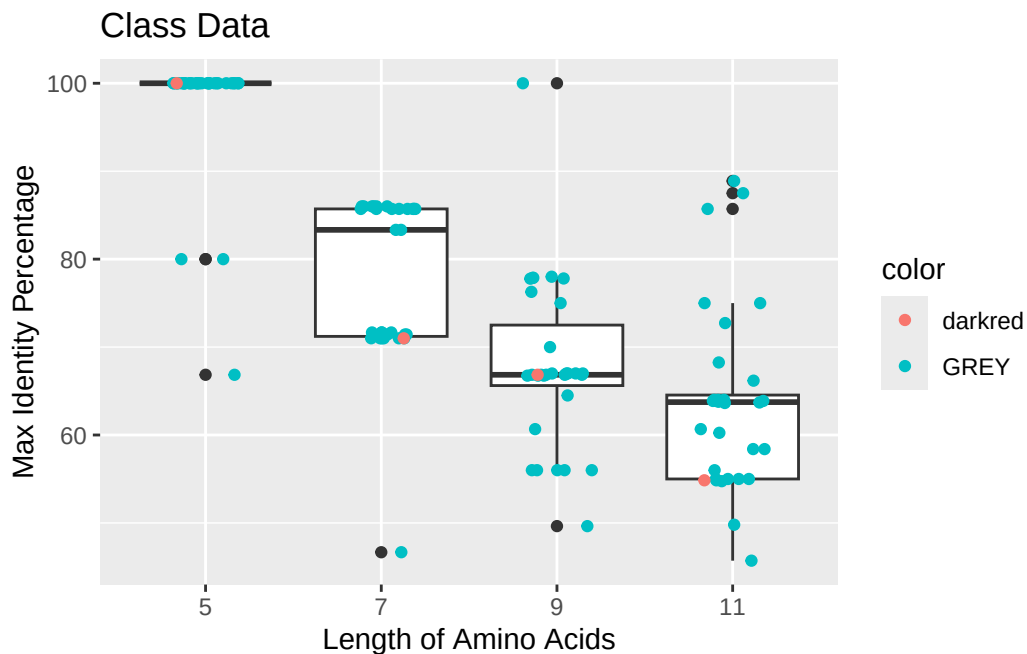
```
color
darkred  GREY
      4    156
```

iv. Now you are ready to remake the plot by adding the following line to your original box plot from Q7: `+ geom_jitter(col=color)`. (Note that the `color` call is not run as an aesthetic here because we are now directly telling ggplot what color to use for each dot). Again, feel free to use the `width` argument to narrow the jitter points if you wish (e.g. `width = 0.2`).


```
# created plot with my differentiated data points
p + geom_jitter(aes(col=color), width=0.2)
```

Warning: Removed 31 rows containing non-finite outside the scale range (``stat_boxplot()``).

Warning: Removed 31 rows containing missing values or values outside the scale range (``geom_point()``).



c. Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s).

Answer: No, all of them are right on the quartiles (1st or 3rd) or the median.

Q9. MHC class II molecules of our immune system display 9 amino acid peptides on the surface of immune cells that, when from a foreign origin, activates a T-cell immune response. Based on your findings in Q7 and 8, speculate why those peptides are 9 amino acids long rather than, say, 5 amino acids.

Answer: Those peptides are most likely 9 amino acids long because they have a much lower median max identity percentage (~65%) than the 5 amino acid peptides (~100%). Therefore, it would be much harder for MHC class II molecules to confuse the foreign particle with our own body cells, since there is a much lower chance of our body having the same 9 amino acid sequence. Thus, they are able to detect the presence of the foreign particle and activate an immune response. However, if the identity percentage is high, in the case of the 5 amino acid peptide, then there is a super high chance that our body contains that sequence, tricking the molecule into thinking that it is not a foreign particle, and failing to activate an immune response.